EXPERIMENT 5

Analysis of Data Communication in a Web Application Using Browser Developer Tools

Subject: No-Code Application Development

Aim

To inspect HTTP/HTTPS communication between a browser and a web application using built-in browser developer tools.

Objectives

- Locate network requests triggered by web-page loading.
- Inspect request methods, status codes, headers, payloads and timing.
- Explain the roles of DNS, TLS, HTTP and browser caching.

Selected Web Application

A publicly accessible website or locally hosted test page inspected using Chrome, Edge or Firefox. Sample values are illustrative; observations will vary by site and browser.

Requirements / Tools

- Desktop browser with developer tools
- Internet access or a local website
- Network tab and optionally a test form or API endpoint

No-Code Connection

When a no-code app loads a form or report, browser requests fetch HTML, CSS, scripts and structured data. Developer tools help confirm whether errors arise from connectivity, permissions or the application configuration.

Illustrative Business Scenario

Open a no-code form preview, submit fictional test values and observe the request and response without recording credentials or personal data.

Experimental Procedure

Follow these steps in the designated simulator, browser or visual app builder. Menu locations and available features may vary by product version and account plan.

1. Open a browser and visit a website permitted for testing. Press F12 or select Inspect.

2. Select the Network tab, enable Preserve log if desired, and reload the page.

3. Identify the main document, stylesheet, JavaScript, images and XHR/fetch resources.

4. Select a request and inspect URL, GET/POST method, response status, content type and headers.

5. Review timing/waterfall information; identify waiting (TTFB), download and any DNS or connection phases shown.

6. Submit a non-sensitive test form if available, observe payload and server response, then compare with a reload using cache disabled.

No-Code Application Perspective

- Identify the user action and the visual interface element involved (form, page, workflow or report).
- Trace the data movement from input to processing to stored record, report or notification.
- Identify configuration that replaces traditional hand-written code and note where scripting might still be required.
- Check whether the experiment can be repeated using sample records and documented settings.

Evidence to Collect

- Screenshot of the configured screen, topology, tool or application.
- A short observation table showing input, action and output.
- One successful test and one failed or exceptional test with expected behavior.

Network panel fields

Field	Meaning	Example
Method	Requested operation	GET or POST
Status	HTTP outcome	200 OK; 404 Not Found
Content-Type	Response format	text/html; application/json
Timing	Phase durations	DNS, connection, waiting, download
Size / Transferred	Data amount and cache effects	Resource and transfer bytes

Sample request record (illustrative)

Request	Method	Response
Main page	GET	200 OK
Missing test asset	GET	404 Not Found
Sample API request	GET	200 OK with JSON

Interpretation / Analysis

- Browser networking is commonly based on HTTPS (HTTP carried over TLS), with DNS resolving the hostname.
- A 200 status indicates a successful HTTP response; 404 means the requested resource was not found.
- Caching and compression can reduce transferred bytes; response times depend on network and server conditions.
- Do not copy authentication cookies, access tokens or personal information into a lab report.

Evaluation Against a No-Code Solution

- Ease of creation: determine how much is configured visually rather than programmed.
- Data and integration: identify where records, connections or services are created and managed.
- Governance: consider role permissions, traceability and safe data handling.
- Limitations: identify any feature that may require paid access, external services or custom logic.

Verification / Test Cases

Suggested Test Record

Test	Input / action	Expected observation
Normal test	Valid sample values and supported action	Expected successful output
Boundary test	Missing value, invalid input or empty data	Validation or clear empty state
Access test	Unapproved user attempts restricted action	Access limited according to role
Repeatability	Re-run same valid example	Consistent output or explain differences

Observation Notes

During the practical session, replace the illustrative outcomes with actual observed results. Include the software/platform name, version if known, screenshots, and any errors encountered.

Results

The Network panel enables investigation of request-response communication, resource loading, response status and performance bottlenecks. Replace illustrative observations with actual screenshots or measurements when performed.

Conclusion

Developer tools make invisible browser-server communication observable and are useful for understanding web protocols, finding errors and improving application performance.

No-Code Learning Outcome

This experiment links the technical concept to a practical no-code application development workflow. It highlights visual configuration, user experience, data handling and safe validation rather than treating the exercise as conventional full-code software development.

Precautions

- Use only fictional sample records for classroom demonstrations; avoid exposing passwords, tokens, employee information or real contact details.
- Record your own screenshots, observed outputs and deviations after executing the experiment. The examples below are models, not verified live results.
- Check access rights before sharing or publishing an app, report, workflow or externally reachable service.